MODUL PRAKTIKUM
PEMROGRAMAN WEB

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
INSTITUT TEKNOLOGI, SAINS, DAN KESEHATAN RS dr. SOEPRAOEN KESDAM V/BRAWIJAYA MALANG
2026

KARTU PESERTA PRAKTIKUM PEMROGRAMAN WEB

Semester Genap Periode 2025/2026
Program Studi Informatika

Nama : Anggun Tri Pratiwi
NIM : 24102026

No.	Praktikum	Tanggal	Nilai	Keterangan
1.	Arsitektur Web (Client-Server, HTTP/HTTPS)			
2.	HTML Semantik dan Form Dasar			
3.	CSS (Selector, Box Model, Responsive)			
4.	JavaScript Dasar dan DOM			
5.	Git/GitHub dan AI-assisted Coding			
6.	PHP Dasar dan Form Handling			
7.	Perancangan Database dan ERD			
8.	Mini Project Frontend (HTML, CSS, JS)			
9.	Perancangan Database dan ERD			
10.	Konsep MVC dan Struktur Laravel			
11.	CRUD Laravel dan Database			

12.	Authentication dan Security Dasar			
13.	REST API (GET dan POST)			
14.	Debugging dan Refactoring			
15.	Final Project Aplikasi Web			

Malang, …….

Disetujui oleh,
Dosen Pengampu .....

Ahsanun Naseh Khudori, M.Kom

Diperiksa oleh,
Asisten Praktikum Informatika

Muhammad Rizky & Bintang Pramudya

DAFTAR ISI

KARTU PESERTA PRAKTIKUM PEMROGRAMAN WEB 2
DAFTAR ISI.....4
1. Mengenal JavaScript.....5
2. Cara Menjalankan JavaScript.....6
3. Sintaks Dasar dan Tipe Data di JavaScript 7
5. Percabangan if (Kondisi Tunggal) 9
6. Percabangan else if dan else (Kondisi Berantai).....9
7. Percabangan else if dan else (Kondisi Berantai).....10
8. Manipulasi DOM (Document Object Model).....10
9. Model Event di JavaScript (Event Handling) 11
10. Pengelolaan Formulir (Form Handling).....11
LATIHAN 4.....12
Langkah 1: Membuat File JavaScript 13
Langkah 2: Berikan id pada tiap elemen di section form 14

PRAKTIKUM 4
JAVASCRIPT
(Javascript Dasar, Manipulasi DOM, Model Event & Pengelolaan Formulir)

TUJUAN

Tujuan dari praktikum ini adalah :

1. Mahasiswa memahami JavaScript sebagai bahasa client-side yang mengontrol perilaku halaman web melalui manipulasi DOM, mencakup konsep variabel, tipe data, struktur kontrol, serta bagaimana script dieksekusi dan terhubung dengan HTML/CSS dalam konteks event-driven
2. Mahasiswa mampu mengimplementasikan event (click, submit, input) secara efisien menggunakan mekanisme event handling modern, termasuk pengelolaan `event object`, pencegahan default behavior, dan kontrol alur interaksi pengguna.
3. Mahasiswa mampu melakukan manipulasi DOM secara dinamis dengan memilih, memodifikasi, serta mengelola struktur elemen dan atribut secara langsung untuk memperbarui tampilan tanpa reload.
4. Mahasiswa mampu merancang validasi form sederhana berbasis JavaScript untuk memastikan konsistensi, kelengkapan, dan format data sebelum diproses lebih lanjut.
5. Mahasiswa mampu menampilkan notifikasi atau feedback interaktif secara real-time tanpa reload halaman untuk meningkatkan responsivitas dan pengalaman pengguna.

DASAR TEORI

1. Mengenal JavaScript

JavaScript (JS) adalah bahasa pemrograman berjenis skrip (scripting language) yang digunakan untuk membuat halaman website menjadi interaktif dan dinamis. Javascript memiliki peran yang sangat luas dalam ekosistem pengembangan web modern, baik disisi klien (browser) maupun sisi server. Berikut kegunaan JavaScript:

a) Client-Side (Di Sisi Klien)

Di sisi klien, JavaScript merupakan peran yang sangat umum karena kode JavaScript dieksekusi langsung di browser pengguna. Beberapa kegunaannya, yaitu:

- **Validasi Formulir**: Mengecek kebenaran data sebelum dikirim ke server.
- **Manipulasi DOM**: Mengubah isi atau gaya halaman web secara dinamis.
- **Interaksi Pengguna**: Merespons aksi pengguna seperti klik, gerakan mouse, atau input keyboard.

b) Server-Side (Di Sisi Server)

Oleh karena lingkungan runtime seperti Node.js, JavaScript kini juga dapat digunakan di sisi server. Kegunaannya, yaitu:

- **Membangun Aplikasi Web**: Mengembangkan backend, API, atau server untuk memproses permintaan klien.
- **Otomatisasi**: Menjalankan skrip untuk otomatisasi tugas harian, misal seperti pengolahan data atau manajemen file.

2. Cara Menjalankan JavaScript

JavaScript dapat dijalankan di berbagai lingkungan, tetapi yang paling umum adalah dijalankan di browser dan di server menggunakan Node.js. Berikut adalah cara-cara dasarnya:

a) Inline JavaScript

Kamu bisa menulis kode JS langsung di dalam atribut HTML. Metode ini tidak disarankan untuk kode yang banyak karena membuat HTML sulit dibaca.

```html
<body>
    <label for="name">Nama Lengkap</label> <br>
    <input type="text" name="nama" id="name">

    <button onclick="alert('Data Berhasil Dikirim')">Submit</button>
</body>

</html>
```

b) Internal JavaScript

Kode JS ditulis di dalam tag <script> yang diletakkan di dalam dokumen HTML, biasanya di bagian <head> atau tepat sebelum tag penutup <body>. Meletakkannya di akhir <body> adalah praktik yang disarankan karena memastikan elemen HTML sudah dimuat sebelum skrip mencoba memanipulasinya.

index.html > html

```html
<!DOCTYPE html>
<html>

<head>
    <title>Halaman Web Interaktif</title>
</head>

<body>
    <h1>Selamat Datang</h1>
    <p id="teks">Teks ini akan diubah oleh JavaScript.</p>

    <script>
        document.getElementById("teks").innerHTML = "Teks ini sudah diubah!";
    </script>
</body>

</html>
```

c) Eksternal JavaScript

Eksternal artinya kamu menulis semua kode JS di dalam berkas terpisah. Misalnya, script.js dan menautkannya ke dokumen HTML menggunakan tag <script> dengan atribut src.

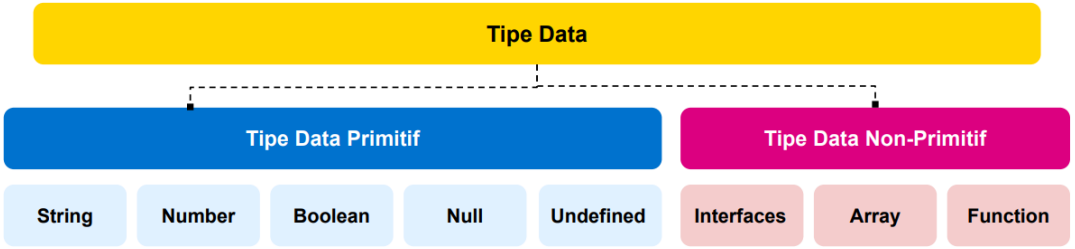
```
index.html > ...
1   <!DOCTYPE html>
2   <html>
3
4   <head>
5       <title>Halaman Web Eksternal</title>
6   </head>
7
8   <body>
9       <h1>Halo!</h1>
10
11      <script src="script.js"></script>
12  </body>
13
14  </html>
15
```

3. Sintaks Dasar dan Tipe Data di JavaScript

Variabel merupakan "wadah" untuk menyimpan data. JavaScript memiliki tiga cara untuk mendeklarasikan variabel, yaitu:

1) **var** : Bisa dideklarasi ulang dan keluar dari blok, sering bermasalah, sebaiknya hindari.
2) **let** : Variabel modern, nilainya bisa dirubah, terbatas pada cakupan blok.
3) **const** : Konstanta, nilainya tetap setelah diberi, paling aman jika tidak berubah.

Adapun, tipe data digunakan untuk menentukan jenis nilai yang dapat disimpan dan diolah dalam sebuah program. Tipe data dibagi menjadi dua, yaitu:



1) **Tipe Data Primitif (**Tipe data ini hanya menyimpan satu nilai.)
   a) **String**
   Tipe data untuk teks. Tipe ini dibuat dengan mengapit teks dalam tanda kutip tunggal ('...') atau ganda ("...")
   b) **Number**
   Tipe data untuk angka, baik bilangan bulat maupun desimal.
   c) **Boolean**
   Tipe data yang hanya memiliki dua nilai, true atau false. Tipe data ini digunakan untuk logika perbandingan.
   d) **Null**
   Merepresentasikan ketiadaan nilai secara sengaja.
   e) **Undifined**
   Variabel yang telah dideklarasikan tetapi belum diberi nilai. JavaScript akan memberinya nilai undefined secara otomatis.
2) **Tipe Data Non-Primitif** (Tipe data ini hanya menyimpan satu nilai.)
   a) **Array**
   Kumpulan data yang terstruktur. Item disimpan dalam urutan tertentu dan dapat diakses menggunakan indeks (dimulai dari 0).
   b) **Object**
   Kumpulan data dalam bentuk pasangan kunci-nilai (key-value pairs).
   c) **Function**
   Kumpulan perintah atau instruksi yang dibungkus dalam satu blok kode agar bisa dijalankan berulang kali.

4. Operator Dasar di JavaScript

Operator merupakan simbol atau tanda khusus yang digunakan untuk melakukan proses perhitungan, perbandingan, maupun logika pada variabel dan nilai. Dengan memahami operator, kamu bisa mengontrol alur logika dan perhitungan dalam program dengan lebih mudah. Berikut jenis-jenis operator dasar yang ada di JavaScript:

Jenis Operator	Fungsi	Contoh
Aritmatika	Melakukan operasi perhitungan matematika	+ (penjumlahan), - (pengurangan), * (perkalian), / (pembagian), % (sisa bagi)
Perbadingan	Membandingkan dua nilai, hasilnya boolean	== (sama dengan, tanpa cek tipe), === (sama dengan dan cek tipe), != (tidak sama dengan), > (lebih besar), < (lebih kecil)

Logika	Menggabungkan atau memanipulasi nilai boolean	&& (AND, true jika keduanya bernilai true) | | (OR, true jika salah satunya benilai true)

5. Percabangan if (Kondisi Tunggal)

Pernyataan if merupakan titik dasar dalam logika bersyarat. If ini berfungsi sebagai penjaga gerbang. Kode di dalamnya hanya akan mendapatkan izin untuk dieksekusi jika dan hanya jika ekspresi di dalam tanda kurung () dievaluasi menjadi nilai Boolean true.

Jika kondisinya adalah false, JavaScript akan mengabaikan seluruh blok kode di dalam kurung kurawal {} dan melanjutkan eksekusi ke baris berikutnya setelah blok if.

```
JS script.js > ...
1   let usia = 16;
2
3   if (usia >= 17) {
4       console.log("Anda boleh membuat KTP");
5   }
6
7   console.log("Program Selesai");
8
```

6. Percabangan else if dan else (Kondisi Berantai)

Ketika program perlu membuat serangkaian keputusan yang saling eksklusif, yaitu hanya satu yang boleh dieksekusi, Kamu perlu menggunakan rantai if...else if...else. Dengan kode tersebut, JavaScript akan mengecek kondisi secara berurutan. Begitu satu kondisi ditemukan true, blok kodenya akan dieksekusi dan seluruh rantai percabangan lainnya diabaikan.

1) else if

Digunakan untuk menguji kondisi kedua, ketiga, dan seterusnya, hanya jika semua kondisi if dan else if sebelumnya gagal (false).

2) else

Bagian opsional ini bertindak sebagai catch-all. Ia dieksekusi hanya jika semua kondisi (if dan else if) di atasnya gagal.

```
JS script.js > ...
let cuaca = "hujan";

if (cuaca === "cerah") {
    console.log("Bawa kacamata hitam");
}

else if (cuaca === "berawan") {
    console.log("Bawa jaket ringan");
}

else {
    console.log("Bawa Payung");
}

```

7. Percabangan else if dan else (Kondisi Berantai)

Perulangan (loops) memungkinkan pengembang untuk mengotomatisasi tugas yang berulang, menghindari penulisan kode yang sama berkali-kali.

a) Perulangan for (Berbasis Hitungan)

Fungsi for loop sering digunakan ketika kamu tahu persis berapa kali kamu perlu mengulang. Strukturnya ringkas dan menempatkan semua kontrol perulangan dalam satu baris. Berikut sintaksnya:

b) Perulangan while (Berbasis Kondisi)

Fungsi while loop lebih fleksibel dan digunakan ketika jumlah iterasi tidak pasti, melainkan bergantung pada pemenuhan suatu kondisi. Perulangan ini akan terus berjalan selama kondisinya tetap true.

8. Manipulasi DOM (Document Object Model)

DOM adalah representasi terstruktur (seperti pohon) dari dokumen HTML kamu. JavaScript menggunakan DOM untuk mengakses, mengubah, menambah, dan menghapus elemen HTML dan konten di dalamnya. Tanpa DOM, JavaScript hanyalah bahasa skrip yang terisolasi.

a) Mengakses Elemen

Langkah pertama dalam manipulasi adalah menemukan elemen yang ingin kamu ubah.

Metode	Deskripsi
document.getElementById('id_name')	Mengakses elemen berdasarkan atribut id-nya (unik). Ini adalah metode tercepat.
document.querySelector('.class_name')	Mengakses elemen pertama yang cocok dengan pemilih CSS (misalnya, .my-class atau #my-id).
document.querySelectorAll('tag_name')	Mengakses semua elemen yang cocok dengan pemilih CSS, mengembalikan NodeList.

b) Mengubah Konten dan Atribut

Setelah mendapatkan elemen, kamu bisa mengubah properti dan isinya.

Properti	Deskripsi
.innerHTML	Mengubah konten di dalam elemen, termasuk tag HTML. Misalnya, membuat teks menjadi tebal
.textContent	Mengubah konten hanya berupa teks. Lebih aman dari serangan cross-site scripting (XSS)
.setAttribute('attr', 'val'	Mengubah nilai atribut elemen. Misalnya, mengubah src pada tag <img>

9. Model Event di JavaScript (Event Handling)

Event merupakan sinyal dari browser bahwa sesuatu telah terjadi. Ini bisa berupa tindakan pengguna (seperti mengklik mouse atau menekan tombol) atau kejadian yang dihasilkan oleh browser (seperti halaman selesai dimuat). Model Event adalah cara JavaScript menangani, mendeteksi, dan merespons peristiwa-peristiwa ini.

a) Konsep Dasar Event Listener

Untuk merespons suatu peristiwa, kamu bisa menggunakan Event Listener. Ini adalah fungsi yang "mendengarkan" suatu objek (misalnya, tombol) dan akan dieksekusi ketika peristiwa tertentu (misalnya, klik) terjadi.

b) Objek Event (event object)

Saat event terjadi, JavaScript secara otomatis meneruskan objek event ke fungsi listener.

10. Pengelolaan Formulir (Form Handling)

Formulir merupakan komponen utama untuk input data pengguna. Pengelolaan formulir di JavaScript melibatkan: (1) mengambil nilai input, (2) validasi data, dan (3) mencegah pengiriman formulir default.

a) **Mengambil Nilai Input**

Nilai dari elemen input, textarea, dan select selalu diakses melalui properti .value

b) **Validasi Sederhana**

Validasi memastikan data yang dimasukkan pengguna memenuhi persyaratan. Validasi sisi klien (di browser) penting untuk pengalaman pengguna yang baik.

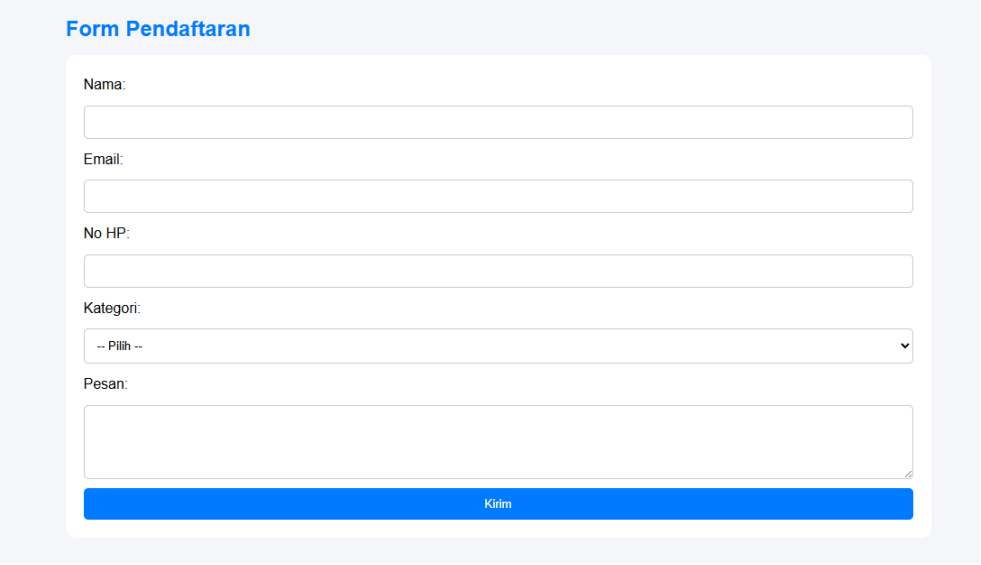
NAMA : **Anggun Tri Pratiwi**
NIM : **24102026**
TANGGAL : **12 April 2026**

LATIHAN 5

Mahasiswa diminta untuk membuat form interaktif menggunakan JavaScript dengan fitur validasi dan manipulasi DOM.
Output Projek yang diharapkan:

Tampilan Form

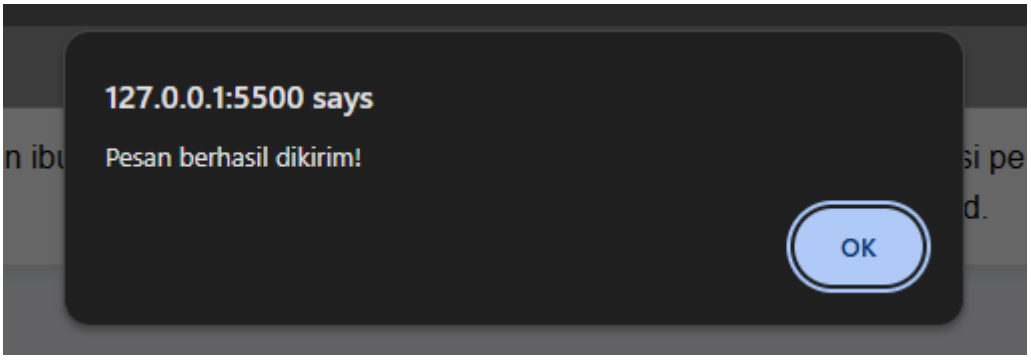
- Input Nama
- Input Email
- Input Pesan
- Tombol Submit



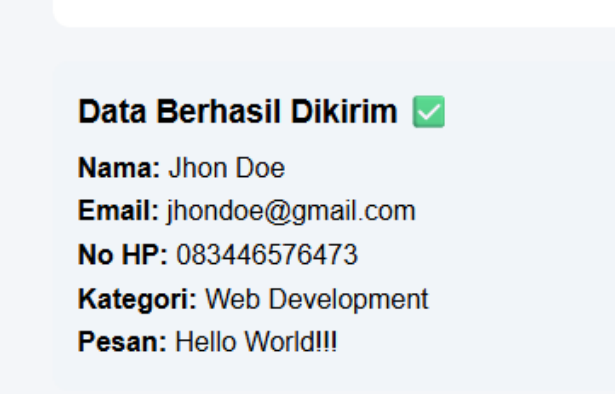
Tampilan Hasil

- Data yang diinput tampil di halaman
- Notifikasi sukses muncul
- Tidak reload halaman

Ada Alert



Keterangan Berhasil Dikirim



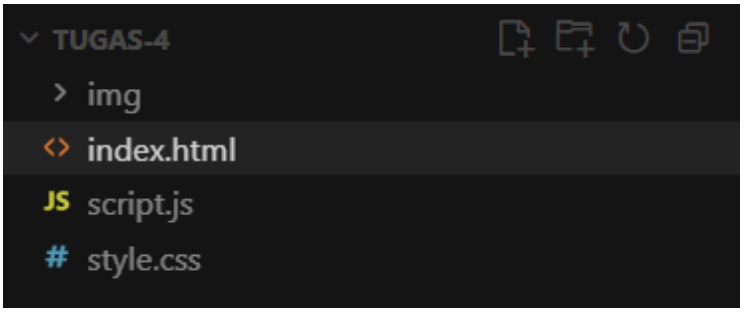
FILE HTML

Gunakan file HTML sebelumnya (LK-03 / LK-04) sebagai dasar, kemudian tambahkan form interaktif.

LANGKAH PRAKTIKUM

Langkah 1: Membuat File JavaScript

Mahasiswa membuat file baru bernama script.js untuk menampung kode JavaScript.



Pada langkah pertama ini buat file script.js di folder root.

Hubungkan ke HTML:

Tambahkan kode berikut sebelum penutup </body>

```html
<script src="script.js"></script>
```

Langkah 2: Berikan id pada tiap elemen di section form

Langkah ini bertujuan agar bisa dipanggila oleh file script.js. Karena harus ada inisialisasi dulu untuk memanggilnya ke script.js

```html
<!-- Form -->
<section id="form">
    <h2>Form Pendaftaran</h2>

    <form id="formPendaftaran">
        <label>Nama:</label>
        <input type="text" id="nama" required>

        <label>Email:</label>
        <input type="email" id="email" required>

        <label>No HP:</label>
        <input type="tel" id="nohp" required>

        <label>Kategori:</label>
        <select id="kategori" required>
            <option value="">-- Pilih --</option>
            <option>Web Development</option>
            <option>Desain</option>
            <option>Konsultasi</option>
        </select>

        <label>Pesan:</label>
        <textarea id="pesan" rows="4"></textarea>

        <button type="submit">Kirim</button>
    </form>

    <!-- OUTPUT -->
    <div id="hasilPendaftaran"></div>
</section>
```

Langkah 3: Mengambil Elemen dari HTML

Pada langkah ini, JavaScript digunakan untuk mengambil elemen form dan area hasil dari HTML menggunakan document.getElementById. Elemen form disimpan dalam variabel form, sedangkan area untuk menampilkan hasil disimpan dalam variabel hasil. Proses ini penting karena JavaScript hanya bisa memanipulasi elemen yang sudah diambil dari DOM.

```javascript
// Ambil elemen
const form = document.getElementById("formPendaftaran");
const hasil = document.getElementById("hasilPendaftaran");
```

Langkah 4: Menangani Event Submit

Pada langkah ini, digunakan event submit untuk menangkap aksi ketika tombol kirim ditekan. Method e.preventDefault() digunakan untuk mencegah reload halaman, sehingga data dapat diproses menggunakan JavaScript tanpa memuat ulang halaman. Ini merupakan dasar dari interaktivitas modern pada web.

```javascript
form.addEventListener("submit", function (e) {
    e.preventDefault();
```

Langkah 5: Mengambil Data Input dari Form

Pada langkah ini, nilai dari setiap input diambil menggunakan .value. Data yang diambil meliputi nama, email, nomor HP, kategori, dan pesan. Data ini nantinya akan digunakan untuk validasi dan ditampilkan kembali ke halaman.

```javascript
const nama = document.getElementById("nama").value;
const email = document.getElementById("email").value;
const nohp = document.getElementById("nohp").value;
const kategori = document.getElementById("kategori").value;
const pesan = document.getElementById("pesan").value;
```

Langkah 6: Validasi Form

Pada langkah ini dilakukan pengecekan apakah ada input yang kosong. Jika ada field yang belum diisi, maka akan muncul notifikasi alert dan proses dihentikan menggunakan return. Validasi ini penting untuk memastikan data yang dikirim lengkap dan tidak kosong.

```javascript
if (nama === "" || email === "" || nohp === "" || kategori === "") {
    alert("Semua field wajib diisi!");
    return;
}
```

Langkah 7: Validasi Email Sederhana

Selain validasi kosong, dilakukan juga validasi format email secara sederhana dengan mengecek apakah email mengandung karakter "@". Jika tidak valid, maka akan muncul alert dan proses dihentikan. Ini merupakan bentuk validasi dasar sebelum data diproses lebih lanjut.

```javascript
if (!email.includes("@")) {
    alert("Email tidak valid!");
    return;
}
```

Langkah 8: Menampilkan Data (Manipulasi DOM)

Pada langkah ini, data yang telah diinput akan ditampilkan kembali ke halaman menggunakan innerHTML. Teknik ini disebut manipulasi DOM, yaitu mengubah isi halaman secara dinamis tanpa reload. Data ditampilkan dalam bentuk card agar lebih rapi dan mudah dibaca.

```javascript
hasil.innerHTML = `
    <div style="margin-top:20px; padding:15px; background:#f1f5f9; border-radius:10px;">
        <h3>Data Berhasil Dikirim ✅</h3>
        <p><strong>Nama:</strong> ${nama}</p>
        <p><strong>Email:</strong> ${email}</p>
        <p><strong>No HP:</strong> ${nohp}</p>
        <p><strong>Kategori:</strong> ${kategori}</p>
        <p><strong>Pesan:</strong> ${pesan}</p>
    </div>
`;
```

Langkah 9: Menampilkan Notifikasi

Pada langkah ini, digunakan alert() untuk memberikan feedback kepada user bahwa data berhasil dikirim. Notifikasi ini penting agar user mengetahui bahwa proses telah selesai dilakukan dengan sukses.

```javascript
alert("Pesan berhasil dikirim!");
```

Langkah 10: Reset Form

Setelah data berhasil dikirim dan ditampilkan, form akan dikosongkan kembali menggunakan form.reset(). Hal ini bertujuan agar form siap digunakan kembali tanpa harus menghapus input secara manual.

```javascript
form.reset();
```

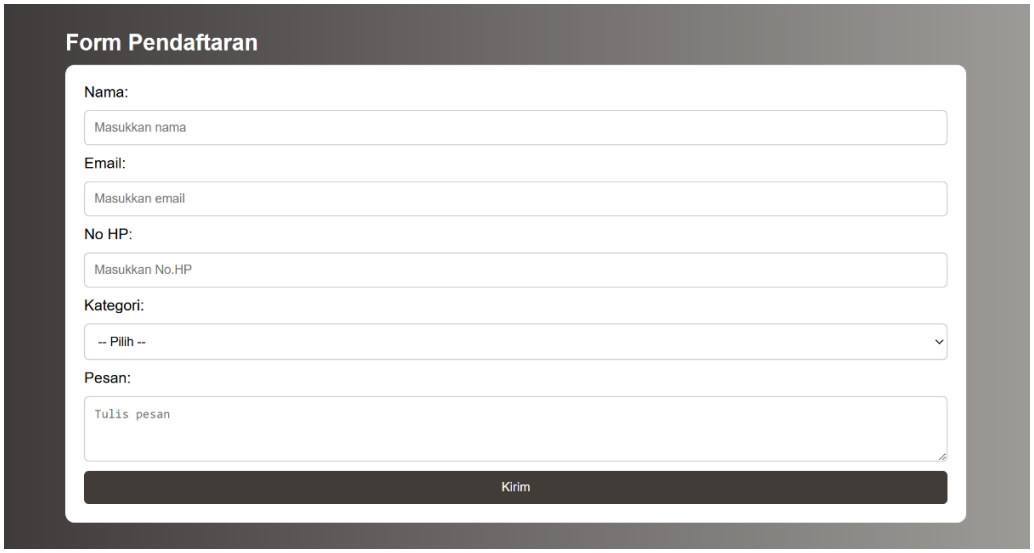
Langkah 11: Event Input (Interaktivitas Tambahan)

Pada langkah terakhir, ditambahkan event input pada field nama. Event ini akan berjalan ketika user sedang mengetik dan menampilkan log di console. Tujuannya adalah untuk menunjukkan bahwa JavaScript dapat merespon aktivitas user secara real-time.

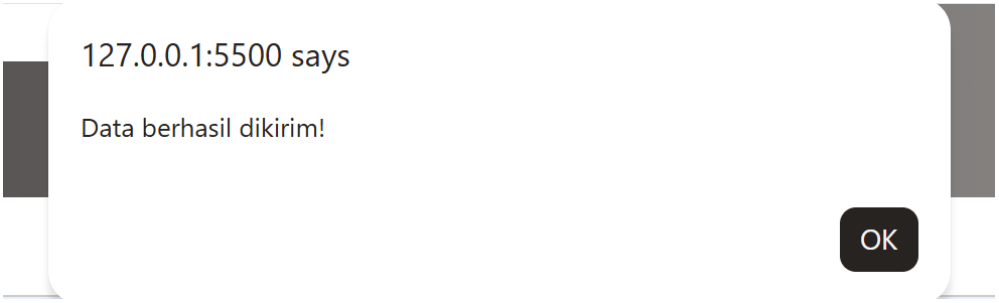
```javascript
document.getElementById("nama").addEventListener("input", function () {
    console.log("User sedang mengetik nama...");
});
```

Screenshot Hasil Tugas Project/Praktikum

Tampilan Form



Tampilan Hasil



Hasil dan Analisa Praktikum

Script ini digunakan untuk mengatur proses pengisian dan pengiriman form pendaftaran menggunakan JavaScript dengan memanfaatkan DOM. Pertama, mengambil elemen dari HTML menggunakan document.getElementById() yaitu form (formPendaftaran) dan area untuk menampilkan hasil (hasilPendaftaran), agar JavaScript berinteraksi langsung dengan elemen HTML.

Ditambahkan event input pada field nama. Event ini akan berjalan setiap kali user mengetik pada input nama, dan menampilkan pesan di console sebagai bentuk interaksi sederhana bahwa user sedang melakukan input. Kemudian, terdapat event submit pada form. Saat tombol kirim ditekan, fungsi e.preventDefault() digunakan agar halaman tidak refresh, sehingga data bisa diproses langsung oleh JavaScript.

Di dalam event submit, program mengambil semua nilai dari input seperti nama, email, nomor HP, kategori, dan pesan. Setelah itu dilakukan pengecekan (validasi), yaitu semua data wajib harus diisi, nama minimal terdiri dari 3 karakter, email harus mengandung tanda “@”. Jika salah satu validasi tidak terpenuhi, maka akan muncul notifikasi alert dan proses dihentikan dengan return.

Jika semua validasi berhasil, maka data yang diinput akan ditampilkan ke halaman menggunakan innerHTML. Ini merupakan bentuk manipulasi DOM, di mana JavaScript menambahkan elemen baru ke dalam halaman tanpa reload. Terakhir, ditampilkan notifikasi sukses menggunakan alert, lalu form akan dikosongkan kembali menggunakan form.reset() agar siap untuk input berikutnya.